

DSA ASSIGNMENT = 1

NAME : SACHIN KUMAR VASHISTHA

UNIVERSITY ROLL NO : 12515001375

CLASS ROLL NO : 58

SECTION: FB

DescriptionEditorialSolutionsAccepted XSubmissions

1464. Maximum Product of Two Elements in an ArraySolved

EasyTopicsCompaniesHint

Given the array of integers `nums`, you will choose two different indices `i` and `j` of that array. Return the maximum value of $(\text{nums}[i]-1) * (\text{nums}[j]-1)$.

Example 1:

Input: `nums = [3,4,5,2]`
Output: 12
Explanation: If you choose the indices `i=1` and `j=2` (indexed from 0), you will get the maximum value, that is, $(\text{nums}[1]-1) * (\text{nums}[2]-1) = (4-1) * (5-1) = 3 * 4 = 12$.

Example 2:

Input: `nums = [1,5,4,5]`
Output: 16
Explanation: Choosing the indices `i=1` and `j=3` (indexed from 0), you will get the maximum value of $(5-1) * (5-1) = 16$.

Example 3:

Input: `nums = [3,7]`
Output: 12

2.8K28427 Online

Code

JavaAuto

```
1 class Solution {
2     public int maxProduct(int[] nums) {
3
4         int n = nums.length;
5         int max = 0;
6
7         for(int i = 0; i < n; i++){
8             for(int j = i + 1; j < n; j++){
9                 max = Math.max(
10                     (nums[i] - 1) * (nums[j] - 1),
11                     max
12                 );
13             }
14         }
15     }
16 }
```

SavedLn 17, Col 1

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2Case 3

Input

nums =
[3,4,5,2]

Output

DescriptionAccepted XEditorialSolutionsSubmissions

162. Find Peak ElementSolved

MediumTopicsCompanies

A peak element is an element that is strictly greater than its neighbors.

Given a **0-indexed** integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [1,2,3,1]`
Output: 2
Explanation: 3 is a peak element and your function should return the index number 2.

Example 2:

Input: `nums = [1,2,1,3,5,6,4]`
Output: 5
Explanation: Your function can return either index number 1 where the peak element is 3, or index number 5 where the peak element is 6.

14.9K564170 Online

Code

JavaAuto

```
1 class Solution {
2     public int findPeakElement(int[] nums) {
3         int low = 0;
4         int high = nums.length - 1;
5
6         while (low < high) {
7             int mid = low + (high - low) / 2;
8
9             if (nums[mid] > nums[mid + 1]) {
10                 high = mid;
11             } else {
12                 low = mid + 1;
13             }
14         }
15
16         return low;
17     }
18 }
```

SavedLn 8, Col 1

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2

Input

nums =
[1,2,3,1]

Output

DescriptionAccepted XEditorialSolutionsSubmissions

33. Search in Rotated Sorted Array

MediumTopicsCompanies

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return the **index of target** if it is in `nums`, or `-1` if it is not in `nums`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`
Output: `4`

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 3`
Output: `-1`

Example 3:

Input: `nums = [1]`, `target = 0`
Output: `-1`

30.6K710☆🔗🔔

459 Online

</>Code

JavaAuto

```
1 class Solution {
2     public int search(int[] nums, int target) {
3         int start = 0, end = nums.length - 1;
4
5         while (start <= end) {
6             int mid = start + (end - start) / 2;
7
8             if (nums[mid] == target) return mid;
9
10            // Check if left part is sorted
11            if (nums[mid] >= nums[start]) {
12                if (target >= nums[start] && target < nums[mid]) {
13                    end = mid - 1; // move left
14                } else {
15                    start = mid + 1; // move right
16                }
17            }
18            // Right part is sorted
19            else {
20                if (target > nums[mid] && target <= nums[end]) {
21                    start = mid + 1; // move right
22                } else {
23                    end = mid - 1; // move left
24                }
25            }
26        }
27        return -1;
28    }
29 }
```

SavedLn 30, Col 2

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2Case 3

Input

DescriptionEditorialSolutionsAccepted XSubmissions

3477. Fruits Into Baskets II

EasyTopicsCompaniesHint

You are given two arrays of integers, `fruits` and `baskets`, each of length `n`, where `fruits[i]` represents the **quantity** of the `ith` type of fruit, and `baskets[j]` represents the **capacity** of the `jth` basket.

From left to right, place the fruits according to these rules:

- Each fruit type must be placed in the **leftmost available basket** with a capacity **greater than or equal** to the quantity of that fruit type.
- Each basket can hold **only one** type of fruit.
- If a fruit type **cannot be placed** in any basket, it remains **unplaced**.

Return the number of fruit types that remain unplaced after all possible allocations are made.

Example 1:

Input: `fruits = [4,2,5]`, `baskets = [3,5,4]`

528182☆🔗🔔

5 Online

</>Code

JavaAuto

```
1 class Solution {
2     public int numOfUnplacedFruits(int[] fruits, int[] baskets) {
3         int n = fruits.length, alloted=0;
4         for(int i=0;i<n;i++)
5         {
6             for(int j=0;j<n;j++)
7             {
8                 if(fruits[i]<=baskets[j])
9                 {
10                    alloted++;
11                    baskets[j] = -1;
12                    break;
13                }
14            }
15        }
16        return n - alloted;
17    }
18 }
```

SavedLn 18, Col 2

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2

[Description](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

121. Best Time to Buy and Sell Stock

Easy | Topics | Companies

You are given an array `prices` where `prices[i]` is the price of a given stock on the `ith` day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return the *maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return `0`.

Example 1:

Input: `prices = [7,1,5,3,6,4]`
Output: `5`
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

36.1K | 861 | 561 Online

</> Code

Java | Auto

```
1 class Solution {
2     public int maxProfit(int[] prices) {
3         int lsf = Integer.MAX_VALUE;
4         int op = 0;
5         int pist = 0;
6
7         for(int i = 0; i < prices.length; i++){
8             if(prices[i] < lsf){
9                 lsf = prices[i];
10            }
11            pist = prices[i] - lsf;
12            if(op < pist){
13                op = pist;
14            }
15        }
16        return op;
17    }
```

Saved | Ln 1, Col 1

☒ Testcase | [Test Result](#)

Accepted | Runtime: 0 ms

☒ Case 1 | ☒ Case 2

[Description](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

1. Two Sum

Easy | Topics | Companies | Hint

You are given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to `target`*.

You may assume that each input would have **exactly one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15], target = 9`
Output: `[0,1]`
Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input: `nums = [3,2,4], target = 6`
Output: `[1,2]`

Example 3:

Input: `nums = [3,3], target = 6`
Output: `[0,1]`

69.5K | 2.1K | 2467 Online

</> Code

Java | Auto

```
1 class Solution {
2     public int[] twoSum(int[] nums, int target) {
3         for(int i=0;i<nums.length;i++){
4             for(int j=i+1;j<nums.length;j++){
5                 if(nums[i] + nums[j] == target){
6                     return new int[] {i,j};
7                 }
8             }
9         }
10        return new int[] {};
11    }
12 }
```

Saved | Ln 1, Col 1

☒ Testcase | [Test Result](#)

Accepted | Runtime: 0 ms

☒ Case 1 | ☒ Case 2 | ☒ Case 3

Input

nums =
[2,7,11,15]

target =

[Description](#) | [Accepted](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

1365. How Many Numbers Are Smaller Than the Current Number

Easy | Topics | Companies | Hint

Given the array `nums`, for each `nums[i]` find out how many numbers in the array are smaller than it. That is, for each `nums[i]` you have to count the number of valid `j`'s such that `j != i` and `nums[j] < nums[i]`.

Return the answer in an array.

Example 1:

Input: `nums = [8,1,2,2,3]`
Output: `[4,0,1,1,3]`
Explanation:
For `nums[0]=8` there exist four smaller numbers than it (1, 2, 2 and 3).
For `nums[1]=1` does not exist any smaller number than it.
For `nums[2]=2` there exist one smaller number than it (1).
For `nums[3]=2` there exist one smaller number than it (1).
For `nums[4]=3` there exist three smaller numbers than it (1, 2 and 2).

Example 2:

Input: `nums = [6,5,4,8]`
Output: `[2,1,0,3]`

6.1K | 88 | 45 Online

</> Code

Java | Auto

```
1 class Solution {
2     public int[] smallerNumbersThanCurrent(int[] nums) {
3
4         int[] buck = new int[101];
5
6         for(int i=0; i<nums.length; i++) {
7             buck[nums[i]] +=1;
8         }
9
10        for(int j=1; j<= 100; j++) {
11            buck[j] += buck[j-1];
12        }
13
14        for(int k=0; k< nums.length; k++) {
15            int pos = nums[k];
16            nums[k] = pos==0 ? 0 : buck[pos-1];
17        }
18
19        return nums;
20    }
```

Saved | Ln 19, Col 21

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

[Description](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

628. Maximum Product of Three Numbers

Easy | Topics | Companies

Given an integer array `nums`, find three numbers whose product is maximum and return the maximum product.

Example 1:

Input: `nums = [1,2,3]`
Output: 6

Example 2:

Input: `nums = [1,2,3,4]`
Output: 24

Example 3:

Input: `nums = [-1,-2,-3]`
Output: -6

4.8K | 223 | 41 Online

</> Code

Java | Auto

```
1 class Solution {
2     public int maximumProduct(int[] nums) {
3         int first_max=-2000,
4             second_max=-2000,
5             third_max=-2000;
6         int min1=2000,min2=2000;
7         for(int i=0;i<nums.length;i++){
8             if(nums[i]>first_max){
9                 third_max=second_max;
10                second_max=first_max;
11                first_max=nums[i];
12            }
13            else if(nums[i]>second_max){
14                third_max=second_max;
15                second_max=nums[i];
16            }
17        }
```

Saved | Ln 5, Col 9

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

26. Remove Duplicates from Sorted Array

Solved

Easy Topics Companies Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [1,1,2]`
Output: `2`, `nums = [1,2,_]`
Explanation: Your function should return `k = 2`, with the first two elements of `nums` being 1 and 2

19.6K 1.1K ☆ 507 Online

<> Code

Java Auto

```
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3
4
5         if(nums.length == 0)
6             return 0;
7
8         int k = 1;
9
10
11         for(int i = 1; i < nums.length; i++){
12
13             if(nums[i] != nums[i-1]){
14                 nums[k] = nums[i];
15                 k++;
16             }
17
18             return k;
19 }
```

Saved

Ln 9, Col 1

Testcase > Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

```
nums =
[1,1,2]
```

Output

```
[1,2]
```